Machine Learning (IC272)

Assignment 3

Support Vector Machine Implementation

Instructor: Dr. Indu Joshi

Institution: IIT Mandi

Date: October 27, 2025

1. Executive Summary

This report presents a comprehensive SVM implementation for term deposit prediction using a banking dataset with 32,950 samples. Multiple SVM variants were implemented from scratch including Linear, Polynomial, and RBF kernels, with systematic hyperparameter tuning.

2. Dataset Overview

The banking dataset contains 15 features and 1 target variable (yes/no for term deposit subscription). The dataset is highly imbalanced with 88.73% negative class and 11.27% positive class, making F1-score the most appropriate evaluation metric.

3. Preprocessing Pipeline

Six critical preprocessing steps were applied: (1) Separate features and target, (2) Encode target to binary, (3) One-hot encode categorical variables (15→48 features), (4) Train-test split (80-20), (5) Feature standardization (mean=0, std=1), (6) Subsampling for kernel methods (5,000 samples).

Why Standardization is Critical

SVM uses distance calculations, making it sensitive to feature scales. Without standardization, features like 'duration' (0-4918) dominate 'age' (17-98). Standardization ensures all features contribute equally and improves convergence speed.

4. SVM Implementation

Implemented three kernel types: Linear ($K=x\cdot x'$), Polynomial ($K=(x\cdot x'+1)^d$), and RBF ($K=\exp(-\gamma||x-x'||^2)$). Each kernel captures different data patterns, from linear separability to complex non-linear relationships.

Hyperparameters

Key parameters tuned: C (regularization, tested [0.01-100]), gamma (RBF influence radius, tested [0.001-1]), and degree (polynomial complexity, tested [2-5]). Grid search performed with 25+ model configurations.

5. Training Results

Linear SVM trained with 5 C values using full dataset. Polynomial SVM tested degrees 2-5 with 5,000 subsample. RBF SVM grid search over 16 C-gamma combinations. Training times: Linear ~5min, Polynomial ~30-40min, RBF ~20min.

Memory Management

Kernel methods require $O(n^2)$ memory. Full dataset kernel matrix: $26{,}360^2 = 5.4$GB. Subsampled to 5,000: 195MB. This trade-off between accuracy and feasibility is essential for large datasets.

6. Evaluation Metrics

Four metrics computed: Accuracy (overall correctness), Precision (TP/(TP+FP)), Recall (TP/(TP+FN)), F1-Score (harmonic mean). F1-score is primary metric due to class imbalance. Confusion matrix provides detailed error analysis.

7. Ensemble Method

Top 3 models combined using majority voting. Each model votes for predicted class; final prediction determined by majority (2 out of 3). Ensemble reduces individual weaknesses and improves robustness.

8. Key Functions

standardize(): Z-score normalization. SimpleSVM class: Dual optimization with kernel support. calculate_metrics(): Computes all evaluation metrics. majority_voting(): Combines model predictions. All implemented using only NumPy, Pandas, Matplotlib, Seaborn.

9. Conclusions

Successfully implemented SVM from scratch without scikit-learn. Trained 25+ models across 3 kernels. Key findings: preprocessing is critical, kernel selection matters, F1-score essential for imbalanced data, subsampling enables large-scale training, ensemble improves stability.

Technical Achievements

✓ Scratch implementation using only allowed libraries ✓ Handled 32,950 samples with memory-efficient techniques ✓ Generated 7 professional visualizations ✓ Comprehensive hyperparameter tuning ✓ Ensemble learning implementation ✓ Complete documentation and code comments